

Using Datasets to load local and remote datasets

You may already know how to download datasets from the Hugging Face Hub using `load_dataset()`. But in real projects, datasets are often **not** on the Hub. Instead, they might be:

- **Local files** on your laptop/PC (CSV, JSON, TXT, etc.)
- **Remote files** on a server, GitHub, or any URL

The good news: **Datasets can load both local and remote files** using the same `load_dataset()` function.

The two key things you provide to `load_dataset()`

1. **The loading script** (tells Datasets the file format), e.g.
 - `"csv"`
 - `"json"`
 - `"text"`
 - `"pandas"`
2. **`data_files`** (the file path(s) or URL(s))

Common examples

```
load_dataset("csv", data_files="my_file.csv")
load_dataset("text", data_files="my_file.txt")
load_dataset("json", data_files="my_file.jsonl")
load_dataset("pandas", data_files="my_dataframe.pkl")
```

Rule of thumb:

The first argument is the format (`"csv"`, `"json"`, etc.), and `data_files` points to your file(s).

Loading a local dataset (SQuAD-it example)

SQuAD-it is an Italian question-answering dataset. The train and test splits are available on GitHub, so you can download them first:

```
wget https://github.com/crux82/squad-it/raw/master/SQuAD_it-train.json.gz
wget https://github.com/crux82/squad-it/raw/master/SQuAD_it-test.json.gz
```

What does **.json.gz** mean?

- It is a **JSON file**
- Compressed using **gzip** (**.gz**)

JSON vs JSON Lines (JSONL)

When loading JSON, you must know whether it's:

1) Ordinary JSON (nested JSON)

- The file contains **one big JSON object/list**
- Can be deeply nested (like dictionaries inside dictionaries)

2) JSON Lines (JSONL)

- **Each line** is a separate JSON object
- Line-separated JSON

SQuAD-it uses **ordinary nested JSON**, and the real dataset content is stored under a key called **"data"**.

Why do we need **field="data"**?

Because the JSON file looks like this conceptually:

```
{  
  "version": "...",  
  "data": [ ... actual examples are here ... ]  
}
```

So you must tell Datasets:

“Please read the dataset from inside the **data** field.”

That's why we load it like this:

```
from datasets import load_dataset
```

```
squad_it_dataset = load_dataset(
```

```
"json",
data_files="SQuAD_it-train.json",
field="data"
)
```

Why does it return a **DatasetDict** and why is the default split **train**?

When you load local files, Datasets typically returns a **DatasetDict**, which is a container for multiple splits.

If you provide only one file, it assumes it is the **train** split by default:

```
DatasetDict({
  train: Dataset(...)
})
```

- **DatasetDict** = holds multiple splits (train/test/validation)
- Only one file given → it becomes **train**

How to inspect one example

```
squad_it_dataset["train"][0]
```

You'll see fields like:

- **title**: topic/article title
- **paragraphs**: list of paragraphs
 - **context**: the passage text
 - **qas**: question-answer items
 - **question**
 - **answers** (answer text + **answer_start**)
 - **id**

This is exactly what you expect for a **QA dataset**.

Why load train + test together?

Because you often want to apply the same preprocessing (cleaning, tokenizing, formatting) to all splits using:

- `Dataset.map()`

It's easier when both `train` and `test` are inside the same `DatasetDict`.

Loading train and test in one `DatasetDict`

You do that by passing a dictionary to `data_files`:

```
from datasets import load_dataset
```

```
data_files = {
    "train": "SQuAD_it-train.json",
    "test": "SQuAD_it-test.json"
}
```

```
squad_it_dataset = load_dataset("json", data_files=data_files, field="data")
```

Now you get:

```
DatasetDict({
  train: Dataset(num_rows=442)
  test: Dataset(num_rows=48)
})
```

`data_files` is flexible

`data_files` can be:

- A single string path
`"file.json"`
- A list of file paths
`["a.json", "b.json"]`
- A dict mapping split name → path
`{"train": "...", "test": "..."}`

You can also use a glob pattern:

```
data_files="*.json"
```

Meaning: “load all JSON files matching this pattern.”

You don't need to manually unzip **.gz** files

Datasets supports **automatic decompression** (GZIP/ZIP/TAR, etc.).

So this works directly:

```
data_files = {  
    "train": "SQuAD_it-train.json.gz",  
    "test": "SQuAD_it-test.json.gz"  
}
```

```
squad_it_dataset = load_dataset("json", data_files=data_files, field="data")
```

No need to run **gzip -d** yourself.

Remote datasets (same idea, but URLs)

To load from GitHub (or any remote server), just put **URLs** inside **data_files** instead of local paths:

```
from datasets import load_dataset
```

```
url = "https://github.com/crux82/squad-it/raw/master/"  
data_files = {  
    "train": url + "SQuAD_it-train.json.gz",  
    "test": url + "SQuAD_it-test.json.gz",  
}
```

```
squad_it_dataset = load_dataset("json", data_files=data_files, field="data")
```

This produces the same **DatasetDict**, and you skip:

- manual downloading
- manual decompression

2. Loading a Remote Dataset

In many companies, datasets are often not stored on your local machine. Instead, they may live on:

- GitHub / S3 / any HTTP server
- An internal company server (where files are accessible via URL)

The advantage of the `datasets` library is that loading remote files is just as easy as loading local files—you simply provide a **URL** in `data_files` instead of a local file path.

Core idea

- Local file: `data_files="SQuAD_it-train.json.gz"`
- Remote file: `data_files="https://.../SQuAD_it-train.json.gz"`

The `datasets` library will automatically:

- Download the file from the URL (and cache it locally)
- Decompress it automatically if it's compressed (e.g., `.gz`)
- Build the dataset object

Example: Loading SQuAD-it from GitHub

```
from datasets import load_dataset
```

```
url = "https://github.com/crux82/squad-it/raw/master/"
```

```
data_files = {  
    "train": url + "SQuAD_it-train.json.gz",  
    "test": url + "SQuAD_it-test.json.gz",  
}
```

```
squad_it_dataset = load_dataset("json", data_files=data_files, field="data")
```

What's happening here?

- `url` is the base URL where the dataset files are hosted
- `data_files` is a dictionary mapping split names (`train`, `test`) to their corresponding URLs

- `field="data"` is required because the dataset is stored inside the nested JSON key called `data`
- `"json"` is the loading script that tells `load_dataset()` the file format is JSON